

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum- 590014, Karnataka.



LAB RECORD

on

Big Data Analytics (23CS6PCBDA)

Submitted by

Varsha Virupaksha Gujjala (1WA23CS034)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU - 560019

January 2026 – May 2026

B.M.S. College of Engineering

Bull Temple Road, Bangalore 560019

(Affiliated to Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Big Data Analytics” carried out by **Varsha Virupaksha Gujjala (1WA23CS034)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfilment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2026. The Lab report has been approved as it satisfies the academic requirements in respect of a Big Data Analytics (23CS6PCBDA) work prescribed for the said degree.

D. Chaitra

Assistant Professor

Department of CSE, BMSCE

Dr. Kavitha Sooda

Professor & HOD

Department of CSE, BMSCE

INDEX

Sl. No.	Date	Experiment Title	Page No.
1	04.03.25	MongoDB- CRUD Operations Demonstration (Practice and Self Study)	1
2	01.04.25	Perform the following DB operations using Cassandra. - Create a keyspace by name Employee - Create a column family by name - Employee-Info with attributes - Emp_Id Primary Key, Emp_Name, Designation, - Date_of_Joining, Salary, Dept_Name - Insert the values into the table in batch - Update Employee name and Department of EmpId 121 - Sort the details of Employee records based on salary - Alter the schema of the table Employee_Info to add a column Projects which stores a set of Projects done by the corresponding Employee. - Update the altered table to add project names. - Create a TTL of 15 seconds to display the values of Employees.	7
3	08.04.25	Perform the following DB operations using Cassandra. - Create a keyspace by name Library - Create a column family by name Library-Info with attributes - Stud_Id Primary Key, - Counter_value of type Counter, - Stud_Name, Book-Name, Book-Id, - Date_of_issue - Insert the values into the table in batch - Display the details of the table created and increase the value of the counter - Write a query to show that a student with id 112 has taken a book "BDA" 2 times. - Export the created column to a csv file g) Import a given csv dataset from local file system into Cassandra column family	10
4	15.04.25	Execution of HDFS Commands for interaction with Hadoop Environment. (Minimum 10 commands to be executed)	17
5	15.04.25	Implement Wordcount program on Hadoop framework	20
6	06.05.25	From the following link extract the weather data https://github.com/tomwhite/hadoop-book/tree/master/input/ncdc/all - Create a MapReduce program to find average temperature for each year from NCDC data set. b) find the mean max temperature for every month	24

7	20.05.25	For a given Text file, Create a Map Reduce program to sort the content in an alphabetic order listing only top 10 maximum occurrences of words.	33
8	20.05.25	Write a Scala program to print numbers from 1 to 100 using for loop.	38
9	20.05.25	Using RDD and FlatMap count how many times each word appears in a file and write out a list of words whose count is strictly greater than 4 using Spark.	39
10	20.05.25	Write a simple streaming program in Spark to receive text data streams on a particular port, perform basic text cleaning (like white space removal, stop words removal, lemmatization, etc.), and print the cleaned text on the screen. (Open Ended Question).	40

Github Link:

Course Outcomes (COs):

CO1	Apply the concept of NoSQL, Hadoop or Spark for a given task
CO2	Analyse big data analytics mechanisms that can be applied to obtain solution for a given problem.
CO3	Design and implement solutions using data analytics mechanisms for a given problem.

LABORATORY PROGRAM – 1

MongoDB- CRUD Operations Demonstration (Practice and Self Study)

COMMAND WITH OUTPUT - USING ATLAS

```
> _MONGOSH
> db["Employee_Info"].find()
<
> use Employee
< switched to db Employee
> db.createCollection("Employee_Info")
< { ok: 1 }
> db.Employee_Info.createIndex({ Emp_Id: 1 }, { unique: true })
O • MongoServerError[DuplicateKey]: Index build failed: 5778370
> db.Employee_Info.insertMany([
  {
    _id: 121,
    Emp_Name: "Alice",
    Designation: "Software Engineer",
    Date_of_Joining: ISODate("2021-06-15"),
    Salary: 75000,
    Dept_Name: "IT"
  },
  {
    _id: 122,
    Emp_Name: "Bob",
    Designation: "Senior Engineer",
    Date_of_Joining: ISODate("2020-03-10"),
    Salary: 90000,
    Dept_Name: "R&D"
  },
  {
    _id: 123,
    Emp_Name: "Charlie",
    Designation: "Manager",
    Date_of_Joining: ISODate("2019-01-20"),
    Salary: 105000,
    Dept_Name: "HR"
  }
])
```

```
< {
  acknowledged: true,
  insertedIds: {
    '0': 121,
    '1': 122,
    '2': 123
  }
}
> db.Employee_Info.updateOne(
  { _id: 121 },
  {
    $set: {
      Emp_Name: "Alice Johnson",
      Dept_Name: "Engineering"
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.Employee_Info.find().sort({ Salary: 1 })
< {
  _id: 121,
  Emp_Name: 'Alice Johnson',
  Designation: 'Software Engineer',
  Date_of_Joining: 2021-06-15T00:00:00.000Z,
  Salary: 75000,
  Dept_Name: 'Engineering'
}
{
  _id: 122,
  Emp_Name: 'Bob',
  Designation: 'Senior Engineer',
```

```

{
  _id: 122,
  Emp_Name: 'Bob',
  Designation: 'Senior Engineer',
  Date_of_Joining: 2020-03-10T00:00:00.000Z,
  Salary: 90000,
  Dept_Name: 'R&D'
}
{
  _id: 123,
  Emp_Name: 'Charlie',
  Designation: 'Manager',
  Date_of_Joining: 2019-01-20T00:00:00.000Z,
  Salary: 105000,
  Dept_Name: 'HR'
}
> db.Employee_Info.find().sort({ Salary: -1 })
< {
  _id: 123,
  Emp_Name: 'Charlie',
  Designation: 'Manager',
  Date_of_Joining: 2019-01-20T00:00:00.000Z,
  Salary: 105000,
  Dept_Name: 'HR'
}
{
  _id: 122,
  Emp_Name: 'Bob',
  Designation: 'Senior Engineer',
  Date_of_Joining: 2020-03-10T00:00:00.000Z,
  Salary: 90000,
  Dept_Name: 'R&D'
}
{
  _id: 121,
  Emp_Name: 'Alice Johnson',
  Designation: 'Software Engineer',

```

```

{
  _id: 121,
  Emp_Name: 'Alice Johnson',
  Designation: 'Software Engineer',
  Date_of_Joining: 2021-06-15T00:00:00.000Z,
  Salary: 75000,
  Dept_Name: 'Engineering'
}
> db.Employee_Info.updateOne(
  { _id: 121 },
  {
    $addToSet: {
      Projects: { $each: ["Payroll System", "Cloud Migration"] }
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.Employee_Info.updateMany(
  {},
  {
    $set: {
      expireAt: new Date(Date.now() + 15000)
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}

```

```

> db.Employee_Info.updateMany(
  {},
  {
    $set: {
      expireAt: new Date(Date.now() + 15000)
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
> db.Employee_Info.updateMany(
  {},
  {
    $set: {
      expireAt: new Date(Date.now() + 15000)
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
> db.Employee_Info.createIndex(
  { expireAt: 1 },
  { expireAfterSeconds: 0 }
)
< expireAt_1
>
Employee>

```


MongoDB- CRUD Operations Demonstration (Practice and Self Study)

COMMAND WITH OUTPUT - USING UBUNTU TERMINAL

```
MyDataBase> use MyDataBase
already on db MyDataBase
MyDataBase> show collections
Customers
NewStudent
Student
MyDataBase> db.Student.find();
[
  {
    _id: 1,
    StudName: 'Michellejacintha',
    Grade: 'VII',
    Hobbies: 'InternetSurfing'
  },
  { _id: 3, Grade: 'VII', StudName: 'AryanDavid', Hobbies: 'Skating' },
  { _id: 2, Grade: 'VIII', StudName: 'Ram', Hobbies: 'Learning' }
]
```

```
test> use MyDataBase
switched to db MyDataBase
MyDataBase> show collections
NewStudent
NewStudent2
Student
MyDataBase> db.NewStudent2.drop();
true
MyDataBase> db.createCollection("Customers");
{ ok: 1 }
MyDataBase> db.Customers.insertMany([{cust_id:1,Balance:200, Type:"S"},]);
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('67d00571207666297fa3b81a') }
}
MyDataBase> db.Customers.insert({cust_id:1,Balance:1000, Type:"Z"})
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('67d0058f207666297fa3b81b') }
}
MyDataBase> db.Customers.insert({cust_id:2,Balance:100, Type:"Z"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('67d0059c207666297fa3b81c') }
}
MyDataBase> db.Customers.insert({cust_id:2,Balance:1000, Type:"C"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('67d005a5207666297fa3b81d') }
}
MyDataBase> db.Customers.insert({cust_id:2,Balance:500, Type:"C"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('67d005ad207666297fa3b81e') }
}
MyDataBase> db.Customers.insert({cust_id:2,Balance:50, Type:"S"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('67d005b2207666297fa3b81f') }
}
MyDataBase> db.Customers.insert({cust_id:3,Balance:500, Type:"Z"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('67d005ba207666297fa3b820') }
}
```

```
MyDataBase> db.Customers.aggregate([
...   {
...     $group: {
...       _id: "$cust_id",           // Group by cust_id
...       minAccBal: { $min: "$Balance" }, // Find the minimum Balance
...       maxAccBal: { $max: "$Balance" } // Find the maximum Balance
...     }
...   }
... ]);
[
  { _id: 3, minAccBal: 500, maxAccBal: 500 },
  { _id: 2, minAccBal: 50, maxAccBal: 1000 },
  { _id: 1, minAccBal: 200, maxAccBal: 1000 }
]
```

```
MyDataBase> db.Customers.aggregate([
...   { $match: { Type: "Z" } },
...   { $group: { _id: "$cust_id", TotAccBal: { $sum: "$Balance" } } },
...   { $match: { TotAccBal: { $gt: 1200 } } }
... ]);
```

```
MyDataBase> db.Customers.aggregate([
...   { $match: { Type: "Z" } },
...   {
...     $group: {
...       _id: "$cust_id",
...       TotAccBal: { $sum: "$Balance" }
...     },
...     {
...       $match: {
...         TotAccBal: { $gt: 200 }
...       }
...     }
...   }
... ]);
[ { _id: 3, TotAccBal: 500 }, { _id: 1, TotAccBal: 1000 } ]
```

```
bmscecse@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC: $ mongoimport --host localhost --db MyDataBase --collection NewStudent2 --type=csv --file /home/bmscecse/Desktop/135.txt --headerline
2025-03-11T14:55:05.192+0530   connected to: mongodb://localhost/
2025-03-11T14:55:05.360+0530   3 document(s) imported successfully. 0 document(s) failed to import.
bmscecse@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC: $ mongoexport --host localhost --db MyDataBase --collection NewStudent2 --type=json --file /home/bmscecse/Desktop/135.txt
2025-03-11T14:55:24.438+0530   error parsing command line options: unknown option "file"
2025-03-11T14:55:24.438+0530   try 'mongoexport --help' for more information
bmscecse@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC: $ mongoexport --host localhost --db MyDataBase --collection NewStudent2 --type=json --out /home/bmscecse/Desktop/135.txt
2025-03-11T14:55:32.771+0530   connected to: mongodb://localhost/
2025-03-11T14:55:32.780+0530   exported 3 records
bmscecse@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC: $
```

LABORATORY PROGRAM – 2

Perform the following DB operations using Cassandra

Questions:

- a) Create a keyspace by name Employee
- b) Create a column family by name
 - Employee-Info with attributes
 - Emp_Id Primary Key, Emp_Name,
 - Designation, Date_of_Joining,
 - Salary, Dept_Name
- c) Insert the values into the table in batch
- d) Update Employee name and Department of Emp-Id 121
- e) Sort the details of Employee records based on salary
- f) Alter the schema of the table Employee_Info to add a column Projects which stores a set of Projects done by the corresponding Employee.
- g) Update the altered table to add project names.
- h) Create a TTL of 15 seconds to display the values of Employees.

COMMAND WITH OUTPUT

```
mongosh mongodb://127.0.0.1:27017/7directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.10
Microsoft Windows [Version 10.0.26100.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\student>mongosh
Current Mongosh Log ID: 6989a2879a3a13ba051e2620
Connecting to:      mongodb://127.0.0.1:27017/7directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.10
Using MongoDB:      8.0.11
Using Mongosh:       2.5.10
Mongosh 2.6.0 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-02-09T08:35:42.510+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use Employee
switched to db Employee
Employee> db.Employee_Info.insertMany([
... {
...   Emp_Id:121,
...   Emp_Name:"Amit",
...   Designation:"Software Engineer",
...   Date_of_Joining:new Date("2019-06-15"),
...   Salary:60000,
...   Dept_Name:"IT"
... },
... {
...   Emp_Id:122,
...   Emp_Name:"Ravi",
...   Designation:"Senior Engineer",
...   Date_of_Joining:new Date("2018-03-20"),
...   Salary:75000,
...   Dept_Name:"R&D"
... },
... {
...   Emp_Id:123,
...   Emp_Name:"Meera",
...   Designation:"Manager",
...   Date_of_Joining:new Date("2016-01-10"),
...   Salary:90000,
...   Dept_Name:"HR"
... }
... ])
acknowledged: true,
insertedIds: {
  '0': ObjectId('6989a28e9a3a13ba051e2621'),
  '1': ObjectId('6989a28e9a3a13ba051e2622'),
}
```

```

mongosh mongodb://127.0.0.1:27020
Employee> db.Employee_Info.find()
[
  {
    _id: ObjectId('6989a2be9a3a13ba051e2621'),
    Emp_Id: 121,
    Emp_Name: 'Amit',
    Designation: 'Software Engineer',
    Date_of_Joining: ISODate('2019-06-15T00:00:00.000Z'),
    Salary: 60000,
    Dept_Name: 'IT'
  },
  {
    _id: ObjectId('6989a2be9a3a13ba051e2622'),
    Emp_Id: 122,
    Emp_Name: 'Ravi',
    Designation: 'Senior Engineer',
    Date_of_Joining: ISODate('2018-03-20T00:00:00.000Z'),
    Salary: 75000,
    Dept_Name: 'R&D'
  },
  {
    _id: ObjectId('6989a2be9a3a13ba051e2623'),
    Emp_Id: 123,
    Emp_Name: 'Meera',
    Designation: 'Manager',
    Date_of_Joining: ISODate('2016-01-10T00:00:00.000Z'),
    Salary: 90000,
    Dept_Name: 'HR'
  }
]
Employee> db.Employee_Info.find().pretty()
{
  "_id" : ObjectId("6989a2be9a3a13ba051e2621"),
  "Emp_Id" : 121,
  "Emp_Name" : "Amit",
  "Designation" : "Software Engineer",
  "Date_of_Joining" : ISODate("2019-06-15T00:00:00.000Z"),
  "Salary" : 60000,
  "Dept_Name" : "IT"
},
{
  "_id" : ObjectId("6989a2be9a3a13ba051e2622"),
  "Emp_Id" : 122,
  "Emp_Name" : "Ravi",
  "Designation" : "Senior Engineer",
  "Date_of_Joining" : ISODate("2018-03-20T00:00:00.000Z"),
  "Salary" : 75000,
  "Dept_Name" : "R&D"
},
{
  "_id" : ObjectId("6989a2be9a3a13ba051e2623"),
  "Emp_Id" : 123,
  "Emp_Name" : "Meera",
  "Designation" : "Manager",
  "Date_of_Joining" : ISODate("2016-01-10T00:00:00.000Z"),
  "Salary" : 90000,
  "Dept_Name" : "HR"
}

```

```

mongosh mongodb://127.0.0.1:27020
Employee> db.Employee_Info.find().pretty()
[
  {
    _id: ObjectId('6989a2be9a3a13ba051e2621'),
    Emp_Id: 121,
    Emp_Name: 'Amit',
    Designation: 'Software Engineer',
    Date_of_Joining: ISODate('2019-06-15T00:00:00.000Z'),
    Salary: 60000,
    Dept_Name: 'IT'
  },
  {
    _id: ObjectId('6989a2be9a3a13ba051e2622'),
    Emp_Id: 122,
    Emp_Name: 'Ravi',
    Designation: 'Senior Engineer',
    Date_of_Joining: ISODate('2018-03-20T00:00:00.000Z'),
    Salary: 75000,
    Dept_Name: 'R&D'
  },
  {
    _id: ObjectId('6989a2be9a3a13ba051e2623'),
    Emp_Id: 123,
    Emp_Name: 'Meera',
    Designation: 'Manager',
    Date_of_Joining: ISODate('2016-01-10T00:00:00.000Z'),
    Salary: 90000,
    Dept_Name: 'HR'
  }
]
Employee> db.Employee_Info.updateOne(
...   {Emp_Id:121},
...   {$set:{Emp_Name:"Amit Kumar", Dept_Name:"Data Science"}}
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
Employee> db.Employee_Info.find({Emp_Id:121})
[
  {
    _id: ObjectId('6989a2be9a3a13ba051e2621'),
    Emp_Id: 121,
    Emp_Name: 'Amit Kumar',
    Designation: 'Software Engineer',
    Date_of_Joining: ISODate('2019-06-15T00:00:00.000Z'),
    Salary: 60000,

```



```

mongosh mongodb://127.0.0.1:27017/
Employee> db.Employee_Info.updateOne(
  .. {Emp_Id:121},
  .. {$set:{Emp_Name:"Amit Kumar", Dept_Name:"Data Science"}}
  .. )
acknowledged: true,
insertedId: null,
matchedCount: 1,
modifiedCount: 1,
upsertedCount: 0

Employee> db.Employee_Info.find({Emp_Id:121})
{
  _id: ObjectId('6989a2be9a3a13ba051e2621'),
  Emp_Id: 121,
  Emp_Name: 'Amit Kumar',
  Designation: 'Software Engineer',
  Date_of_Joining: ISODate('2019-06-15T00:00:00.000Z'),
  Salary: 60000,
  Dept_Name: 'Data Science'
}

Employee> db.Employee_Info.find().sort({Salary:-1})
{
  _id: ObjectId('6989a2be9a3a13ba051e2623'),
  Emp_Id: 123,
  Emp_Name: 'Neera',
  Designation: 'Manager',
  Date_of_Joining: ISODate('2016-01-10T00:00:00.000Z'),
  Salary: 90000,
  Dept_Name: 'HR'
},
{
  _id: ObjectId('6989a2be9a3a13ba051e2622'),
  Emp_Id: 122,
  Emp_Name: 'Ravi',
  Designation: 'Senior Engineer',
  Date_of_Joining: ISODate('2018-03-20T00:00:00.000Z'),
  Salary: 75000,
  Dept_Name: 'R&D'
},
{
  _id: ObjectId('6989a2be9a3a13ba051e2621'),
  Emp_Id: 121,
  Emp_Name: 'Amit Kumar',
  Designation: 'Software Engineer',
  Date_of_Joining: ISODate('2019-06-15T00:00:00.000Z'),
  Salary: 60000,

```

```

mongosh mongodb://127.0.0.1
employee> db.Employee_Info.updateOne(
..   {Emp_Id:121},
..   {$set:{
..     Projects:["AI-Analytics","Cloud-Migration","BigData-Pipeline"]
..   }}
.. )
..

acknowledged: true,
insertedId: null,
matchedCount: 1,
modifiedCount: 1,
upsertedCount: 0

employee> db.Employee_Info.find().sort({Salary:1})

{
  _id: ObjectId('6989a2be9a3a13ba051e2621'),
  Emp_Id: 121,
  Emp_Name: 'Amit Kumar',
  Designation: 'Software Engineer',
  Date_of_Joining: ISODate('2019-06-15T00:00:00.000Z'),
  Salary: 60000,
  Dept_Name: 'Data Science',
  Projects: [ 'AI-Analytics', 'Cloud-Migration', 'BigData-Pipeline' ]
},
{
  _id: ObjectId('6989a2be9a3a13ba051e2622'),
  Emp_Id: 122,
  Emp_Name: 'Ravi',
  Designation: 'Senior Engineer',
  Date_of_Joining: ISODate('2018-03-20T00:00:00.000Z'),
  Salary: 75000,
  Dept_Name: 'R&D'
},
{
  _id: ObjectId('6989a2be9a3a13ba051e2623'),
  Emp_Id: 123,
  Emp_Name: 'Neera',
  Designation: 'Manager',
  Date_of_Joining: ISODate('2016-01-10T00:00:00.000Z'),
  Salary: 90000,
  Dept_Name: 'HR'
}

employee> db.Employee_Info.insertOne({
..   Emp_Id:130,
..   Emp_Name:"TemporaryUser",
..   createdAt:new Date()
.. })
..

```

```

Emp_Id: 122,
Emp_Name: 'Ravi',
Designation: 'Senior Engineer',
Date_of_Joining: ISODate('2018-03-20T00:00:00.000Z'),
Salary: 75000,
Dept_Name: 'R&D'
},
{
  _id: ObjectId('6989a2be9a3a13ba051e2623'),
  Emp_Id: 123,
  Emp_Name: 'Heera',
  Designation: 'Manager',
  Date_of_Joining: ISODate('2016-01-10T00:00:00.000Z'),
  Salary: 90000,
  Dept_Name: 'HR'
}
]
Employee> db.Employee_Info.insertOne({
...   Emp_Id:130,
...   Emp_Name:'TemporaryUser',
...   createdAt:new Date()
... })
...
{
  acknowledged: true,
  insertedId: ObjectId('6989a3909a3a13ba051e2624')
}
Employee> db.Employee_Info.createIndex(
...   {createdAt:1},
...   {expireAfterSeconds:15}
... )
...
createdAt_1
Employee> db.Employee_Info.find({Emp_Id:130})
[
  {
    _id: ObjectId('6989a3909a3a13ba051e2624'),
    Emp_Id: 130,
    Emp_Name: 'TemporaryUser',
    createdAt: ISODate('2026-02-09T09:06:24.471Z')
  }
]
Employee> show dbs
Employee      108.00 KiB
Employee_Info  72.00 KiB
admin         40.00 KiB
config        108.00 KiB
e_commerce_db 120.00 KiB
local         96.00 KiB
travelmate    40.00 KiB
Employee>

```

LABORATORY PROGRAM – 3

Perform the following DB operations using Cassandra

Questions:

- Create a keyspace by name Library
- Create a column family by name Library-Info with attributes
 - Stud_Id Primary Key,
 - Counter_value of type Counter,
 - Stud_Name, Book-Name, Book-Id,
 - Date_of_issue
- Insert the values into the table in batch
- Display the details of the table created and increase the value of the counter
- Write a query to show that a student with id 112 has taken a book “BDA” 2 times.
- Export the created column to a csv file
- Import a given csv dataset from local file system into Cassandra column family

COMMAND WITH OUTPUT

```
cqlsh> CREATE KEYSPACE Library
... WITH replication = {
...     'class': 'SimpleStrategy',
...     'replication_factor': 1
... };
```

```
cqlsh> USE Library;
```

```
cqlsh:library> CREATE TABLE Library_Info (
...     Stud_Id int,
...     Stud_Name text,
...     Book_Name text,
...     Book_Id int,
...     Date_of_issue date,
...     PRIMARY KEY (Stud_Id, Book_Id)
... );
cqlsh:library> CREATE TABLE Library_Counter (
...     Stud_Id int,
...     Book_Name text,
...     Counter_value counter,
...     PRIMARY KEY (Stud_Id, Book_Name)
... );
```



```
cqlsh:library> BEGIN BATCH
...
... INSERT INTO Library_Info (Stud_Id, Stud_Name, Book_Name, Book_Id,
Date_of_issue)
... VALUES (112, 'Rahul', 'BDA', 101, '2026-02-15');
...
... INSERT INTO Library_Info (Stud_Id, Stud_Name, Book_Name, Book_Id,
Date_of_issue)
... VALUES (113, 'Amit', 'DBMS', 102, '2026-02-14');
...
... APPLY BATCH;
```

```
cqlsh:library> UPDATE Library_Counter
... SET Counter_value = Counter_value + 1
... WHERE Stud_Id = 112 AND Book_Name = 'BDA';
cqlsh:library> UPDATE Library_Counter
... SET Counter_value = Counter_value + 1
... WHERE Stud_Id = 112 AND Book_Name = 'BDA';
```

```
cqlsh:library> SELECT * FROM Library_Info;
```

stud_id	book_id	book_name	date_of_issue	stud_name
113	102	DBMS	2026-02-14	Amit
112	101	BDA	2026-02-15	Rahul

(2 rows)

```
cqlsh:library> SELECT * FROM Library_Counter;
```

stud_id	book_name	counter_value
112	BDA	2

```
cqlsh:library> SELECT Counter_value
... FROM Library_Counter
... WHERE Stud_Id = 112 AND Book_Name = 'BDA';
```

counter_value
2

```
Starting copy of library.library_info with columns [stud_id, book_id, book_name,  
date_of_issue, stud_name].
```

```
Processed: 2 rows; Rate:      14 rows/s; Avg. rate:      14 rows/s
```

```
2 rows exported to 1 files in 0.169 seconds.
```

```
cqlsh:library> COPY Library_Counter TO 'library_counter.csv' WITH HEADER = TRUE;  
Using 16 child processes
```

```
Starting copy of library.library_counter with columns [stud_id, book_name, count  
er_value].
```

```
Processed: 1 rows; Rate:      10 rows/s; Avg. rate:      10 rows/s
```

```
1 rows exported to 1 files in 0.120 seconds.
```

```
cqlsh:library>
```

```
cqlsh:library> COPY Library_Info  
... (stud_id, book_id, book_name, date_of_issue, stud_name)  
... FROM 'library_info.csv'  
... WITH HEADER = TRUE;
```

```
Using 16 child processes
```

```
Starting copy of library.library_info with columns [stud_id, book_id, book_name,  
date_of_issue, stud_name].
```

```
Processed: 2 rows; Rate:      4 rows/s; Avg. rate:      5 rows/s
```

```
2 rows imported from 1 files in 0.387 seconds (0 skipped).
```

```
cqlsh:library> SELECT * FROM Library_Info;
```

stud_id	book_id	book_name	date_of_issue	stud_name
113	102	DBMS	2026-02-14	Anit
112	101	BDA	2026-02-15	Rahul

```
(2 rows)
```

LABORATORY PROGRAM – 4

Execution of HDFS Commands for interaction with Hadoop Environment. (Minimum 10 commands to be executed)

COMMAND WITH OUTPUT

```
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC: $ cd ./Desktop/
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC: ~/Desktop$ start-all.sh
WARNING: Attempting to start all Apache Hadoop daemons as hadoop in 10 seconds.
WARNING: This is not a recommended production deployment configuration.
WARNING: Use CTRL-C to abort.
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bmscscse-HP-Elite-Tower-800-G9-Desktop-PC]
Starting resourcemanager
Starting nodemanagers
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -mkdir /Lab05
```

```
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Hadoop
ls: '/Hadoop': No such file or directory
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Lab05
```

```
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ touch test.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ nano test.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -put ./text.txt /Lab05/text.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Lab05
Found 1 items
-rw-r--r-- 1 hadoop supergroup          19 2024-05-13 14:33 /Lab05/text.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -cat /Lab05/text.txt
Hello
How are you?
```

```
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Lab05
Found 2 items
-rw-r--r-- 1 hadoop supergroup          15 2024-05-13 14:40 /Lab05/test.txt
-rw-r--r-- 1 hadoop supergroup          19 2024-05-13 14:33 /Lab05/text.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -getmerge /Lab05 /text.txt /Lab05 /test.txt ../
Downloads/Merged.txt
getmerge: '/text.txt': No such file or directory
getmerge: '/test.txt': No such file or directory
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -getmerge /Lab05/text.txt /Lab05/test.txt ../Do
wnloads/Merged.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -getfacl /Lab05
# file: /Lab05
# owner: hadoop
# group: supergroup
user::rwx
group::r-x
other::r-x
```

```
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -put /home/hadoop/Desktop/Welcome.txt /abc/WC.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -copyFromLocal /home/hadoop/Desktop/Welcome.txt /abc/WC.txt
copyFromLocal: '/abc/WC.txt': File exists
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -get /abc/WC.txt /home/hadoop/Downloads/WMC.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -getmerge /abc/ /home/hadoop/Desktop/Merge.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -getfacl /abc/
# file: /abc
# owner: hadoop
# group: supergroup
user::rwx
group::r-x
other::r-x
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -cat /abc/WC.txt
hello world
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -mv /abc /FFF
hadoop fs -ls /FFF
Found 3 items
-rw-r--r-- 1 hadoop supergroup          12 2025-04-15 14:53 /FFF/WC.txt
-rw-r--r-- 1 hadoop supergroup          12 2024-05-14 14:35 /FFF/File.txt
-rw-r--r-- 1 hadoop supergroup          12 2024-05-14 14:38 /FFF/file_cp_local.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -cp /CSE/ /LLL
```



```
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -copyToLocal /Lab05/text.txt ../Documents
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -copyToLocal /Lab05/text.txt ../Documents
```

```
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -cat /Lab05/text.txt
Hello
How are you?
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -mv /Lab05 /test_Lab05
```

```
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -ls /test_Lab05
Found 2 items
-rw-r--r-- 1 hadoop supergroup 15 2024-05-13 14:40 /test_Lab05/test.txt
-rw-r--r-- 1 hadoop supergroup 19 2024-05-13 14:33 /test_Lab05/text.txt
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -cp /test_Lab05/ /Lab05
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -ls /Lab05
Found 2 items
-rw-r--r-- 1 hadoop supergroup 15 2024-05-13 14:51 /Lab05/test.txt
-rw-r--r-- 1 hadoop supergroup 19 2024-05-13 14:51 /Lab05/text.txt
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -ls /test_Lab05
Found 2 items
-rw-r--r-- 1 hadoop supergroup 15 2024-05-13 14:40 /test_Lab05/test.txt
-rw-r--r-- 1 hadoop supergroup 19 2024-05-13 14:33 /test_Lab05/text.txt
```

LABORATORY PROGRAM – 5

Implement Wordcount program on Hadoop framework

CODE, COMMAND WITH OUTPUT

Driver Code

```
// Importing libraries
import java.io.IOException;

import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class WCDriver extends Configured implements Tool {

    public int run(String[] args) throws IOException {
        if (args.length < 2) {
            System.out.println("Please give valid inputs");
            return -1;
        }

        JobConf conf = new JobConf(WCDriver.class);
        conf.setJobName("WordCount");

        FileInputFormat.setInputPaths(conf, new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new Path(args[1]));

        conf.setMapperClass(WCMapper.class);
        conf.setReducerClass(WCReducer.class);

        conf.setMapOutputKeyClass(Text.class);
        conf.setMapOutputValueClass(IntWritable.class);

        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);

        JobClient.runJob(conf);
        return 0;
    }

    // Main Method
    public static void main(String[] args) throws Exception {
        int exitCode = ToolRunner.run(new WCDriver(), args);
        System.out.println("Job Exit Code: " + exitCode);
    }
}
```

Mapper Code

```
// Importing libraries
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.OutputCollector;
```

```
import org.apache.hadoop.mapred.Reporter;
```



```

public class WCMapper extends MapReduceBase implements Mapper<LongWritable, Text, Text, IntWritable> {

    // Map function
    public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable> output, Reporter reporter)
        throws IOException {
        String line = value.toString();

        // Splitting the line on whitespace
        for (String word : line.split("\\s+")) {
            if (word.length() > 0) {
                output.collect(new Text(word), new IntWritable(1));
            }
        }
    }
}

```

Reducer Code

```

// Importing libraries
import java.io.IOException;
import java.util.Iterator;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;

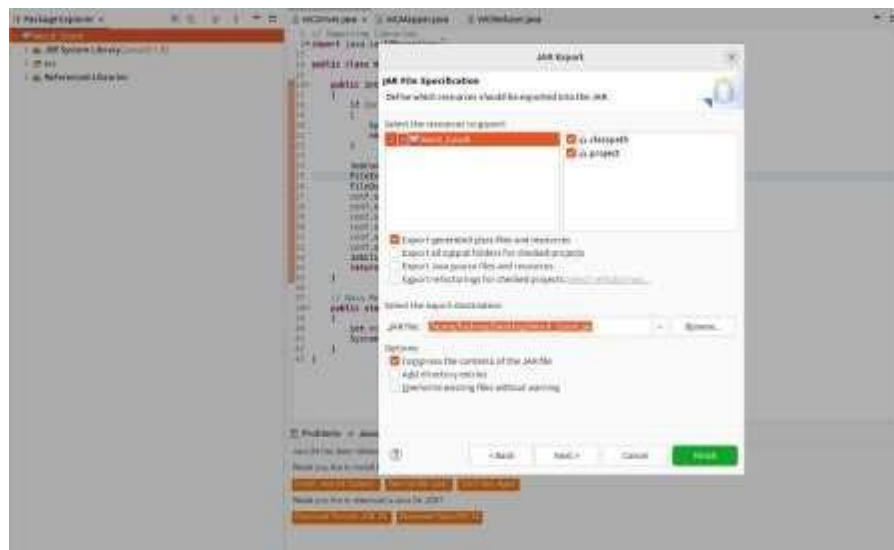
public class WCReducer extends MapReduceBase implements Reducer<Text, IntWritable, Text, IntWritable> {

    // Reduce function
    public void reduce(Text key, Iterator<IntWritable> values,
        OutputCollector<Text, IntWritable> output,
        Reporter reporter) throws IOException {
        int count = 0;

        // Counting the frequency of each word
        while (values.hasNext()) {
            count += values.next().get();
        }

        output.collect(key, new IntWritable(count));
    }
}

```



```

hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~$ cd ./Desktop/
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ start-all.sh
WARNING: Attempting to start all Apache Hadoop daemons as hadoop in 10 seconds.
WARNING: This is not a recommended production deployment configuration.
WARNING: Use CTRL-C to abort.
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bmscscse-HP-Elite-Tower-800-G9-Desktop-PC]
Starting resourcemanager
Starting nodemanagers
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -mkdir /Lab06
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Lab06

```

```

hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ jps
7360 DataNode
7928 ResourceManager
8681 Jps
7178 NameNode
8091 NodeManager
7644 SecondaryNameNode
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ cd ..
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~$ cd ./Desktop/
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ nano file1.txt

```

```

hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -copyFromLocal -f /home/hadoop/Desktop/file1.txt /rgs/test.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop jar /home/hadoop/Desktop/WordCount.jar wordcount.WordCount /rgs/test.txt /output
JAR does not exist or is not a normal file: /home/hadoop/Desktop/WordCount.jar
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop jar /home/hadoop/Desktop/Word_Count.jar wordcount.WordCount /rgs/test.txt /output
Exception in thread "main" java.lang.ClassNotFoundException: wordcount.WordCount
    at java.base/java.net.URLClassLoader.findClass(URLClassLoader.java:476)
    at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:594)
    at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:527)
    at java.base/java.lang.Class.forName0(Native Method)
    at java.base/java.lang.Class.forName(Class.java:398)
    at org.apache.hadoop.util.RunJar.run(RunJar.java:321)
    at org.apache.hadoop.util.RunJar.main(RunJar.java:241)
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -cat /output/part-00000
are 1
brother 1
family 1
hi 1
how 5
is 4
job 1
sister 1
you 1
your 4
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /output
Found 2 items
-rw-r--r-- 1 hadoop supergroup 0 2024-05-21 15:21 /output/_SUCCESS
-rw-r--r-- 1 hadoop supergroup 69 2024-05-21 15:21 /output/part-00000

```

LABORATORY PROGRAM – 6

Implement Weather program on Hadoop framework

Questions:

From the following link extract the weather data

<https://github.com/tomwhite/hadoopbook/tree/master/input/ncdc/all>

- a) Create a MapReduce program to find average temperature for each year from NCDC data set.
- b) find the mean max temperature for every month.

CODE, COMMAND WITH OUTPUT – A

Driver Code

```
package temp;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AverageDriver {

    public static void main(String[] args) throws Exception {

        if (args.length != 2) {
            System.err.println("Please enter both input and output parameters.");
            System.exit(-1);
        }

        // Creating a configuration and job instance
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Average Calculation");

        job.setJarByClass(AverageDriver.class);

        // Input and output paths
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        // Setting mapper and reducer classes
        job.setMapperClass(AverageMapper.class);
        job.setReducerClass(AverageReducer.class);

        // Output key and value types
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        // Submitting the job and waiting for it to complete
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

}

Mapper Code

```
package temp;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class AverageMapper extends Mapper<LongWritable, Text, Text, IntWritable> {

    public static final int MISSING = 9999;

    @Override
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString();

        // Extract year from fixed position
        String year = line.substring(15, 19);
        int temperature;

        // Determine if there's a '+' sign
        if (line.charAt(87) == '+') {
            temperature = Integer.parseInt(line.substring(88, 92));
        } else {
            temperature = Integer.parseInt(line.substring(87, 92));
        }

        // Quality check character
        String quality = line.substring(92, 93);

        // Only emit if data is valid
        if (temperature != MISSING && quality.matches("[01459]")) {
            context.write(new Text(year), new IntWritable(temperature));
        }
    }
}
```

Reducer Code

```
package temp;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class AverageReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    public void reduce(Text key, Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException {

        int sumTemp = 0;
        int count = 0;

        for (IntWritable value : values) {
            sumTemp += value.get();
            count++;
        }

        if (count > 0) {
            int average = sumTemp / count;
            context.write(key, new IntWritable(average));
        }
    }
}
```

Name	Size	Type	Modified
META-INF	25 bytes	Folder	
.classpath	2.2 kB	unknown	06 May 2025, 14:40
.project	377 bytes	unknown	06 May 2025, 14:34
AverageDriver.class	1.6 kB	Java class	06 May 2025, 14:42
AverageMapper.class	2.4 kB	Java class	06 May 2025, 14:42
AverageReducer.class	2.3 kB	Java class	06 May 2025, 14:42

[illegible]


```

2025-05-06 14:59:24,581 INFO mapreduce.Job: Counters: 36
  File System Counters
    FILE: Number of bytes read=153118
    FILE: Number of bytes written=1493804
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=1776380
    HDFS: Number of bytes written=8
    HDFS: Number of read operations=15
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=4
    HDFS: Number of bytes read erasure-coded=0
  Map-Reduce Framework
    Map input records=6565
    Map output records=6564
    Map output bytes=59076
    Map output materialized bytes=72210
    Input split bytes=103
    Combine input records=0
    Combine output records=0
    Reduce input groups=1
    Reduce shuffle bytes=72210
    Reduce input records=6564
    Reduce output records=1
    Spilled Records=13128
    Shuffled Maps =1
    Failed Shuffles=0
    Merged Map outputs=1
    GC time elapsed (ms)=0
    Total committed heap usage (bytes)=1266679808
  Shuffle Errors
    BAD_ID=0
    CONNECTION=0
    IO_ERROR=0
    WRONG_LENGTH=0
    WRONG_MAP=0
    WRONG_REDUCE=0
  File Input Format Counters
    Bytes Read=888190
  File Output Format Counters
    Bytes Written=8

```

```

    Bytes Written=8
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -ls /weather
Found 2 items
drwxr-xr-x  - hadoop supergroup          0 2025-05-06 14:59 /weather/output
-rw-r--r--  1 hadoop supergroup    888190 2025-05-06 14:50 /weather/test.txt
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -ls /weather/output
Found 2 items
-rw-r--r--  1 hadoop supergroup          0 2025-05-06 14:59 /weather/output/_SUCCESS
-rw-r--r--  1 hadoop supergroup          8 2025-05-06 14:59 /weather/output/part-r-00000
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -cat /weather/output/part-r-00000
1901    46
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ █

```

CODE, COMMAND WITH OUTPUT – B

Driver Code

```
package meanmax;

import org.apache.hadoop.conf.Configuration; import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable; import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat; import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MeanMaxDriver {

    public static void main(String[] args) throws Exception { if (args.length != 2) {

        System.err.println("Please enter both input and output parameters."); System.exit(-
        1);

    }

    Configuration conf = new Configuration();

    Job job = Job.getInstance(conf, "Mean and Max Temperature");
    job.setJarByClass(MeanMaxDriver.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    job.setMapperClass(MeanMaxMapper.class);
    job.setReducerClass(MeanMaxReducer.class);

    job.setOutputKeyClass(Text.class); job.setOutputValueClass(IntWritable.class);

    System.exit(job.waitForCompletion(true) ? 0 : 1);

    }

}
```

Mapper Code

```
package meanmax;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
```

```

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Mapper;

public class MeanMaxMapper extends Mapper<LongWritable, Text, Text, IntWritable> {

    public static final int MISSING = 9999;

    @Override
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString();

        // Extract month from positions 19-20
        String month = line.substring(19, 21);
        int temperature;

        // Extract temperature considering optional '+'
        if (line.charAt(87) == '+') {
            temperature = Integer.parseInt(line.substring(88, 92));
        } else {
            temperature = Integer.parseInt(line.substring(87, 92));
        }

        // Quality check
        String quality = line.substring(92, 93);

        if (temperature != MISSING && quality.matches("[01459]")) {
            context.write(new Text(month), new IntWritable(temperature));
        }
    }
}

```

Reducer Code

```

package meanmax;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class MeanMaxReducer extends Reducer<Text, IntWritable, Text, Text> {

    @Override
    public void reduce(Text key, Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException {

        int sumTemp = 0;
        int count = 0;
        int maxTemp = Integer.MIN_VALUE;

        for (IntWritable value : values) {
            int temp = value.get();
            sumTemp += temp;
            count++;

            if (temp > maxTemp) {
                maxTemp = temp;
            }
        }

        if (count > 0) {
            int avgTemp = sumTemp / count;
            String result = "mean=" + avgTemp + " max=" + maxTemp;
            context.write(key, new Text(result));
        }
    }
}

```

[illegible]

```

2025-05-06 15:26:36,233 INFO mapreduce.Job: Counters: 36
  File System Counters
    FILE: Number of bytes read=126934
    FILE: Number of bytes written=1466688
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=1776388
    HDFS: Number of bytes written=74
    HDFS: Number of read operations=15
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=4
    HDFS: Number of bytes read erasure-coded=0
  Map-Reduce Framework
    Map input records=6565
    Map output records=6564
    Map output bytes=45948
    Map output materialized bytes=59082
    Input split bytes=95
    Combine input records=0
    Combine output records=0
    Reduce input groups=12
    Reduce shuffle bytes=59082
    Reduce input records=6564
    Reduce output records=12
    Spilled Records=13128
    Shuffled Maps=1
    Failed Shuffles=0
    Merged Map outputs=1
    GC time elapsed (ms)=0
    Total committed heap usage (bytes)=1052770304
  Shuffle Errors
    BAD_ID=0
    CONNECTION=0
    IO_ERROR=0
    WRONG_LENGTH=0
    WRONG_MAP=0
    WRONG_REDUCE=0
  File Input Format Counters
    Bytes Read=888190
  File Output Format Counters
    Bytes Written=74

```

```

hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hdfs dfs -cat /out8/*
01      4
02      0
03      7
04     44
05    100
06    168
07    219
08    198
09    141
10    100
11     19
12      3

```

LABORATORY PROGRAM – 7

For a given Text file, Create a Map Reduce program to sort the content in an alphabetic order listing only top 10 maximum occurrences of words.

CODE, COMMAND WITH OUTPUT

Driver Code (TopNDriver.java)

```
package samples.topn;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class TopNDriver {

    public static void main(String[] args) throws Exception {
        if (args.length != 3) {
            System.err.println("Usage: TopNDriver <in> <temp-out> <final-out>");
            System.exit(2);
        }

        Configuration conf = new Configuration();

        // === Job 1: Word Count ===
        Job wcJob = Job.getInstance(conf, "word count");
        wcJob.setJarByClass(TopNDriver.class);
        wcJob.setMapperClass(WordCountMapper.class);
        wcJob.setCombinerClass(WordCountReducer.class);
        wcJob.setReducerClass(WordCountReducer.class);
        wcJob.setOutputKeyClass(Text.class);
        wcJob.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(wcJob, new Path(args[0]));
        Path tempDir = new Path(args[1]);
        FileOutputFormat.setOutputPath(wcJob, tempDir);

        if (!wcJob.waitForCompletion(true)) {
            System.exit(1);
        }

        // === Job 2: Top N ===
        Job topJob = Job.getInstance(conf, "top 10 words");
        topJob.setJarByClass(TopNDriver.class);
        topJob.setMapperClass(TopNMapper.class);
        topJob.setReducerClass(TopNReducer.class);
        topJob.setMapOutputKeyClass(IntWritable.class);
        topJob.setMapOutputValueClass(Text.class);
        topJob.setOutputKeyClass(Text.class);
        topJob.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(topJob, tempDir);
        FileOutputFormat.setOutputPath(topJob, new Path(args[2]));

        System.exit(topJob.waitForCompletion(true) ? 0 : 1);
    }
}
```


}

Reducer Code (WordCountReducer.java)

```
package samples.topn;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WordCountReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}
```

Reducer Code (TopNReducer.java)

```
package samples.topn;

import java.io.IOException;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class TopNReducer
    extends Reducer<IntWritable, Text, Text, IntWritable> {

    // TreeMap with descending order of keys (counts)
    private TreeMap<Integer, List<String>> countMap =
        new TreeMap<>(Collections.reverseOrder());

    @Override
    protected void reduce(IntWritable key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {

        int cnt = key.get();
        List<String> words = countMap.getOrDefault(cnt, new ArrayList<>());
        for (Text w : values) {
            words.add(w.toString());
        }
        countMap.put(cnt, words);
    }

    @Override
    protected void cleanup(Context context)
        throws IOException, InterruptedException {

        // collect top 10 word→count pairs
        List<WordCount> topList = new ArrayList<>();
        int seen = 0;
        for (Map.Entry<Integer, List<String>> entry : countMap.entrySet()) {
```

```

        int cnt = entry.getKey();
        for (String w : entry.getValue()) {
            topList.add(new WordCount(w, cnt));
            seen++;
            if (seen == 10) break;
        }
    if (seen == 10) break;
    }

    // sort these 10 entries alphabetically by word
    Collections.sort(topList, (a, b) -> a.word.compareTo(b.word));

    // emit final top 10 in alphabetical order
    for (WordCount wc : topList) {
        context.write(new Text(wc.word), new IntWritable(wc.count));
    }
}

// helper class
private static class WordCount {
    String word;
    int count;
    WordCount(String w, int c) { word = w; count = c; }
}

```

```

C:\hadoop-3.3.0\sbin>jps
11072 DataNode
20528 Jps
5620 ResourceManager
15532 NodeManager
6140 NameNode

C:\hadoop-3.3.0\sbin>hdfs dfs -mkdir /input_dir

C:\hadoop-3.3.0\sbin>hdfs dfs -ls /
Found 1 items
drwxr-xr-x - Anusree supergroup          0 2021-05-08 19:46 /input_dir

C:\hadoop-3.3.0\sbin>hdfs dfs -copyFromLocal C:\input.txt /input_dir

C:\hadoop-3.3.0\sbin>hdfs dfs -ls /input_dir
Found 1 items
-rw-r--r--  1 Anusree supergroup        36 2021-05-08 19:48 /input_dir/input.txt

C:\hadoop-3.3.0\sbin>hdfs dfs -cat /input_dir/input.txt
hello
world
hello
hadoop
bye

```

```

C:\hadoop-3.3.0\sbin>hadoop jar C:\sort.jar samples.topn.TopN /input_dir/input.txt /output_dir
2021-05-08 19:54:54,582 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2021-05-08 19:54:55,291 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/Anusree/.staging/job_1620483374279_0001
2021-05-08 19:54:55,821 INFO input.FileInputFormat: Total input files to process : 1
2021-05-08 19:54:56,261 INFO mapreduce.JobSubmitter: number of splits:1
2021-05-08 19:54:56,552 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1620483374279_0001
2021-05-08 19:54:56,552 INFO mapreduce.JobSubmitter: Executing with tokens: []
2021-05-08 19:54:56,843 INFO conf.Configuration: resource-types.xml not found
2021-05-08 19:54:56,843 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2021-05-08 19:54:57,387 INFO impl.YarnClientImpl: Submitted application application_1620483374279_0001
2021-05-08 19:54:57,587 INFO mapreduce.Job: The url to track the job: http://LAPTOP-JG329ESD:8088/proxy/application_1620483374279_0001/
2021-05-08 19:54:57,588 INFO mapreduce.Job: Running job: job_1620483374279_0001
2021-05-08 19:55:13,792 INFO mapreduce.Job: Job job_1620483374279_0001 running in uber mode : false
2021-05-08 19:55:13,794 INFO mapreduce.Job:  map 0% reduce 0%
2021-05-08 19:55:20,020 INFO mapreduce.Job:  map 100% reduce 0%
2021-05-08 19:55:27,116 INFO mapreduce.Job:  map 100% reduce 100%
2021-05-08 19:55:33,199 INFO mapreduce.Job: Job job_1620483374279_0001 completed successfully
2021-05-08 19:55:33,334 INFO mapreduce.Job: Counters: 54
File System Counters:
  FILE: Number of bytes read=65
  FILE: Number of bytes written=530397
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=142
  HDFS: Number of bytes written=31
  HDFS: Number of read operations=8
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=2
  HDFS: Number of bytes read erasure-coded=0

```

```

C:\hadoop-3.3.0\sbin>hdfs dfs -cat /output_dir/*
hello      2
hadoop     1
world      1
bye        1

C:\hadoop-3.3.0\sbin>

```

LABORATORY PROGRAM – 8

Write a Scala program to print numbers from 1 to 100 using for loop.

CODE, COMMAND WITH OUTPUT

```
object PrintNumbers {  
  def main(args: Array[String]): Unit = {  
  
    for(i <- 1 to 100) {  
      println(i)  
    }  
  
  }  
}
```

```
macos@macos:~/Elite-Tower-800-G4-Desktop-PC: $ spark-shell  
25/05/20 11:28:13 WARN Utils: Your hostname, macos@Elite-Tower-800-G4-Desktop-PC resolves to a loopback address: 127.0.0.1; using 10.124.3.88 instead (on interface eno1)  
25/05/20 11:28:13 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address.  
WARNING: An illegal reflective access operation has occurred  
WARNING: Illegal reflective access by org.apache.spark.unsafe.Platform (file:/opt/spark/jars/spark-unsafe_2.12-3.0.3.jar) to constructor java.nio.DirectByteBuffer(long, int)  
WARNING: Please consider reporting this to the maintainers of org.apache.spark.unsafe.Platform  
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations  
WARNING: All illegal access operations will be denied in a future release  
25/05/20 11:28:13 WARN NativeCodeLoader: Unable to load native-boost library for your platform.... using builtin java classes where applicable  
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties  
Setting default log level to "WARN".  
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).  
Spark context Web UI available at http://10.124.3.88:4040  
Spark context available as 'sc' (master = local[*], app-id = local-1747720690918).  
Spark session available as 'spark'.  
Welcome to  
  
      ____  
     / ___/____  ____  
    / __/ __  / __  /  
   /___/____/____/   version 3.0.3  
  
Using Scala version 2.12.10 (OpenJDK 64-Bit Server VM, Java 11.0.20)  
Type in expressions to have them evaluated.  
Type :help for more information.  
  
scala> for (i <- 1 to 100) println i  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 6  
5 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
```


LABORATORY PROGRAM – 9

Using RDD and FlatMap count how many times each word appears in a file and write out a list of words whose count is strictly greater than 4 using Spark.

CODE, COMMAND WITH OUTPUT

```
val textFile = sc.textFile("/home/bhoom/Desktop/wc.txt")

val counts = textFile.flatMap(line => line.split(" ")).map(word => (word,1)).reduceByKey(_
+ _)

import scala.collection.immutable.ListMap

val sorted=ListMap(counts.collect.sortWith(_._2 > _._2):_*)// sort in descending order
based on values

println(sorted)

for((k,v)<-sorted) {
  if(v>4) {
    print(k+",")
    print(v)
    println()
  }
}
```

```
scala> val rdd = spark.sparkContext.textFile("file:/home/bmscecse/Desktop/scala")
rdd: org.apache.spark.rdd.RDD[String] = file:/home/bmscecse/Desktop/scala MapPartitionsRDD[1] at textFile at <console>:23

scala> val counts = rdd.flatMap(_.split("\\s+")).map(word => (word.toLowerCase, 1)).reduceByKey(_ + _).filter(_._2 > 4)
counts: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[5] at filter at <console>:25

scala> counts.collect().foreach{ case (word, count) => println(s"$word $count") }
spark 6

scala>
```

LABORATORY PROGRAM – 10

Write a simple streaming program in Spark to receive text data streams on a particular port, perform basic text cleaning (like white space removal, stop words removal, lemmatization, etc.), and print the cleaned text on the screen. (Open Ended Question).

CODE, COMMAND WITH OUTPUT

```
# Install NLTK and download required data (run once)
!pip install nltk

import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, regexp_replace, split, explode, udf
from pyspark.sql.types import ArrayType, StringType
from pyspark.ml.feature import StopWordsRemover
from nltk.stem import WordNetLemmatizer

# Initialize SparkSession
spark = SparkSession.builder.appName("TextProcessing").getOrCreate()

# Define your input lines
lines = [
    "Hello, I hate you.",
    "I hate that I love you.",
    "Don't want to, but I can't put",
    "nobody else above you."
]

# Create DataFrame from lines
df = spark.createDataFrame(lines, "string").toDF("value")

# Step 1: Lowercase and remove punctuation
df_clean = df.select(regexp_replace(lower(col("value")), "[^a-zA-Z\\s]", "").alias("cleaned"))

# Step 2: Tokenize the cleaned text
df_tokens = df_clean.select(split(col("cleaned"), "\\s+").alias("tokens"))

# Step 3: Remove stop words
remover = StopWordsRemover(inputCol="tokens", outputCol="filtered")
df_filtered = remover.transform(df_tokens)

# Step 4: Lemmatization using NLTK WordNetLemmatizer with UDF
lemmatizer = WordNetLemmatizer()

def lemmatize_words(words):
    return [lemmatizer.lemmatize(word) for word in words]

lemmatize_udf = udf(lemmatize_words, ArrayType(StringType()))

df_lemmatized = df_filtered.withColumn("lemmatized", lemmatize_udf(col("filtered")))
```

Step 5: Explode the lemmatized words and show results

```
df_lemmatized.select(explode(col("lemmatized")).alias("word")).show(truncate=False)
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nltk) (8.2.0)
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from nltk) (1.5.0)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.11/dist-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from nltk) (4.67.1)
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
+-----+
|word |
+-----+
|hello |
|hate |
|hate |
|love |
|dont |
|want |
|cant |
|put |
|nobody|
|else |
+-----+
```